

Team Delivery Package: Dashboard & Reporting

Enterprise Milestone-Based Technical Specifications

MODULE NAME:	Team 7: Dashboard & Reporting
GIT BRANCH:	feature/reporting-dashboard
LEAD MEMBERS:	Hazem, Mahmoud Maher
WORKLOAD TIME:	3 weeks
KICKOFF DATE:	11 July 2026
CODE FREEZE:	23 July 2026
VERSION:	v1.2.0

1. Cover Page & Metadata Summary

This delivery package establishes the contract specifications for the **Team 7: Dashboard & Reporting**.

Milestone	Target Date	Status	Description
Project Kickoff	11 July 2026	COMPLETED	Milestone scope locked & repository structured.
Code Freeze	23 July 2026	IN PROGRESS	All functional packages and tests locked.
Integration Phase	24 July 2026	PLANNED	Multi-team pipeline joins and bug resolution.
Final Presentation	25 July 2026	PLANNED	Final delivery and platform presentation.

2. Team Overview & Assignments

Resource Details

Lead Team Members: **Hazem, Mahmoud Maher**
Branch Space: ``feature/reporting-dashboard``
Role Assignment: **Frontend Architects & Report Designers**
Difficulty Rating: **Medium** | Priority Index: **Critical** | Est. Workload: **3 weeks**

3. Business & Technical Goals

Business Objective

Present the loaded dataset, data profiles, and models in a user-friendly UI.

Technical Objective

Establish streamlit components, manage parameters sidebars, and write markdown reports.

Position inside AutoAnalyst AI

User Interaction and Presentation layer.

4. System Architecture & folder layout

Assigned folder Tree Structure

```
app/  
■■■ streamlit_app.py  
src/autoanalyst/  
■■■ reporting/  
    ■■■ __init__.py  
    ■■■ report_generator.py
```

Target Code Modules

Your codebase modifications belong inside: `app/streamlit\_app.py, src/autoanalyst/reporting/report\_generator.py`.

Functional Code Interface Specifications

```
# src/autoanalyst/reporting/report_generator.py
from pathlib import Path

def create_markdown_report(title: str, insights: list[str], output_path: str) -> Path:
    """Generates a formatted markdown file detailing key insights."""
    pass
```

API Specification

```
create_markdown_report(title: str, insights: List[str], output_path: str) -> Path
Returns:
- Path object confirming the physical file output destination.
```

5. Git Workflow & Code Integration Policy

Branching Workflow

All developers pull and write inside `feature/reporting-dashboard` created from the latest `develop` branch.

Pull Request Requirements

1. Merge latest develop branch changes into features before raising PRs.
2. Assign team lead reviews and resolve conflicts.
3. A minimum of one approved review check is required.

6. Quality Standards & Guidelines

Format & Lint Rules

All variables, methods, and file layouts must use lower_case snake_case parameters. Code comments must be in English.

Error Checking & Inputers

All functional blocks should be guarded by custom exception checks (raise ValueError/KeyError).

Logging Rules

Incorporate logging warnings rather than std print statements.

7. Expected Deliverables & Verification

Required Files list

`app/streamlit\_app.py, src/autoanalyst/reporting/report\_generator.py`

Verification Code Assertions

Unit test validations are situated in: `tests/test\_reporting.py`.

Pipeline integration assertions belong in: `tests/test\_pipeline.py`.

IMPORTANT GUIDELINE

All functional checks must compile successfully with zero syntax warnings before code freeze.

8. Definition of Done Checklist

- [] All functions compile cleanly with PEP 8 checks.
- [] Unit tests cover all key methods and functions.
- [] Try-except checks isolate external dataframe issues safely.
- [] The PR has been reviewed and verified by Team 1.

9. Common Mistakes & Troubleshooting

Pitfalls to Avoid

- Adding machine learning algorithms or feature cleaning directly inside Streamlit pages.
- Using hardcoded output directory paths, causing path errors on other OS platforms.
- Failing to use Streamlit session state, causing UI reset issues during button clicks.